~ Basic Notes • Step by Step ~

Web Development

Basics • Diagrams • All Important Topics

Index

1 What is Web Development?
2 How the Web Works
3 Websites & Web Apps
4 The Three Building Blocks
5 Inside the Browser
6 Tools You Need
7 HTTP Basics
8 Web Architecture & Rendering
9 Front-end Basics
10 Back-end Basics
11 Authentication & Sessions
12 Databases
13 APIs & JSON
14 Real-time Web & PWA
15 Git & GitHub
16 Package Managers & Build Tools
17 Web Security Basics
18 Performance & SEO
19 Testing & Debugging
20 Deployment & Hosting
21 Roadmap & Key Takeaways
22 Web Terms Glossary
23 Interview Q&A
24 Cheat Sheet

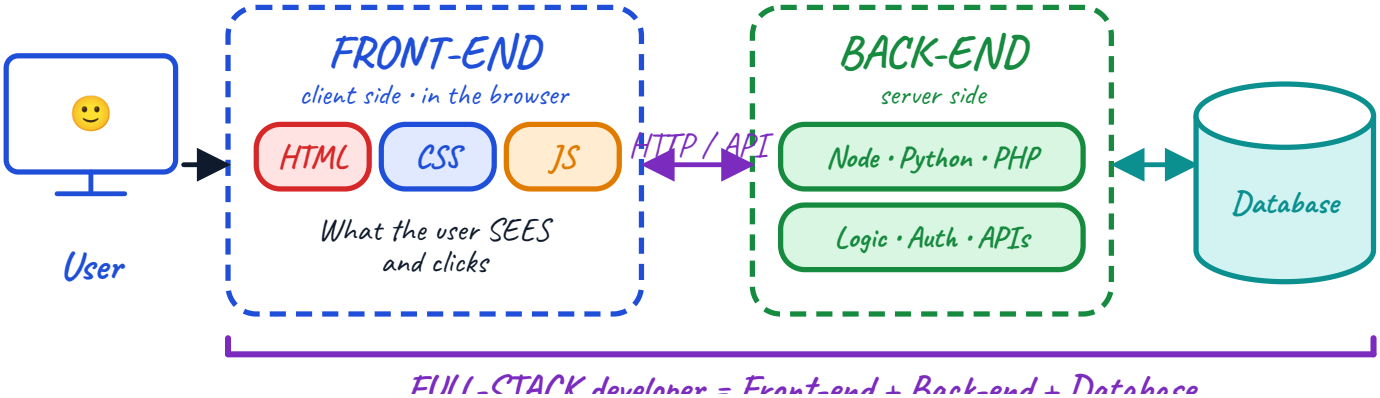
How to use

→ Every chapter has a **diagram** – draw it yourself in your notebook to remember it!
→ Yellow highlight = most important point • *red* = tags • *blue* = attributes / commands.

Notes by asadokz18 ✿

1. What is Web Development?

- **Web development** = building, creating and maintaining **websites and web applications** that run in a web browser (Google, YouTube, Daraz, Facebook are all built by web developers).
- It includes: **web design, front-end coding, back-end coding, databases, testing, deployment and maintenance.**



Type	What it is	Main skills / tools
Front-end	Everything the user **sees and touches**: layout, colours, buttons, forms, animations. Runs in the **browser** (client side).	HTML, CSS, JavaScript, React, Vue, Tailwind
Back-end	The **hidden brain**: processes requests, applies rules, talks to the database, sends data back. Runs on the **server**.	Node.js, Python, PHP, Java, SQL, APIs
Full-stack	Developer who can work on **both** front-end and back-end (and database + deployment).	MERN / MEAN / Django + React…

Restaurant analogy 🍽

Front-end
Dining hall: menu, tables, decoration – what the customer sees.

Back-end
Kitchen: chefs cook the order and follow recipes.

Database
Store room: all ingredients (data) are kept here.

Other roles in web development

- **UI / UX designer** – designs the look & experience (Figma)
- **DevOps engineer** – deployment, servers, CI/CD
- **QA / tester** – finds bugs
- **Database administrator** – manages data
- **Security engineer** – protects the site
- **Project manager** – plans, deadlines

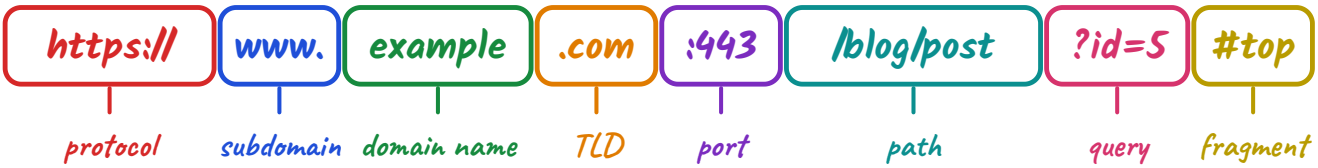
Why learn it?
High demand · work from anywhere · freelancing · build your own products · creative + logical work.

2. How the Web Works

Client & Server

- → **Client** = the device / browser that **asks** for something (your phone, laptop, Chrome).
- → **Server** = a powerful always-on computer that **stores websites and answers** requests.
- → They talk using the **request - response** model over the **Internet** (network of networks). **Web ≠ Internet**: the Internet is the network; the Web is a service (pages) that runs on it.

URL – Uniform Resource Locator (web address)



DNS & IP address

IP address

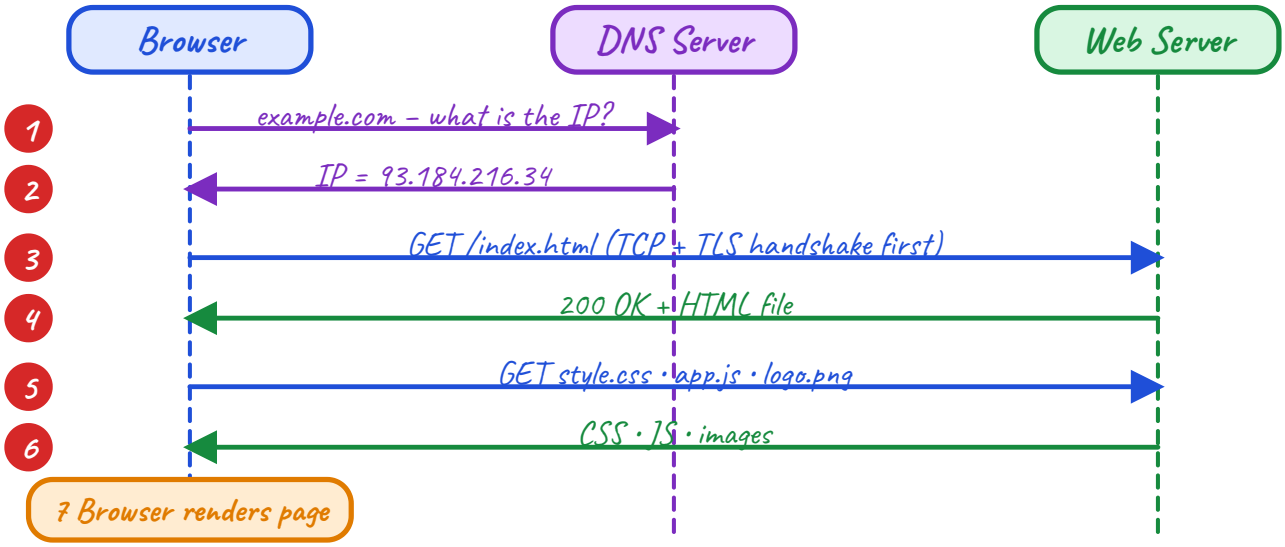
Unique number of every device: **93.184.216.34** (IPv4) or **2606:2800:...** (IPv6). Computers use numbers, not names.

DNS (Domain Name System)

The **phone book of the Internet**: converts ***example.com*** → IP address. Results are cached by browser, OS and ISP.

DNS records: ***A*** (domain → IPv4) · ***AAAA*** (IPv6) · ***CNAME*** (alias to another name) · ***MX*** (mail server) · ***TXT*** (verification) · ***NS*** (name servers).

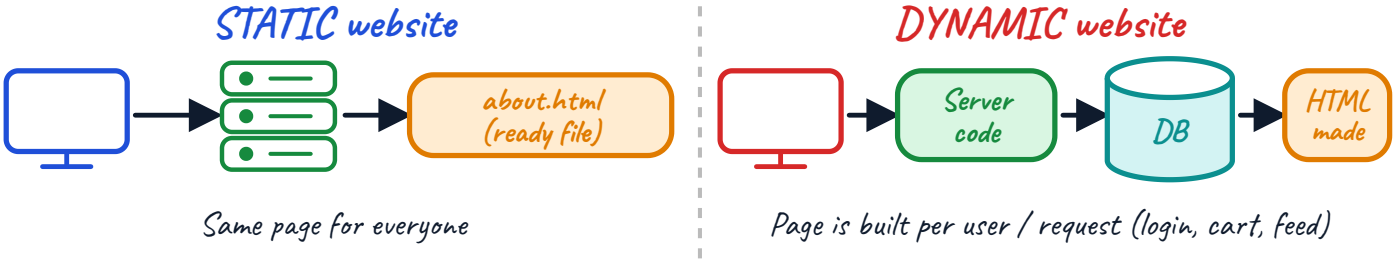
What happens when you open a website? (step by step)



Also: browser first checks its cache; **TCP** connection (3-way handshake) + **TLS** (HTTPS encryption) happen before the request; the server may query a database; assets load in parallel; JS then runs.

3. Websites & Web Apps

Static vs Dynamic website



	Static website	Dynamic website / web app
Content	Fixed HTML files – same for every visitor	Generated by server code per request / user
Built with	HTML + CSS + (some JS)	+ back-end language + database
Examples	Portfolio, landing page, docs, blog (Jekyll / Hugo)	Facebook, Gmail, Daraz, YouTube, online banking
Speed & cost	Very fast, cheap, secure	Slower, costs more, more to secure
User login / data	No	Yes

→ **Website** = mostly information to read (news, company). **Web app** = interactive software in the browser (Google Docs, Trello). **PWA** = web app that installs like a mobile app.

Domain & Hosting

Domain name

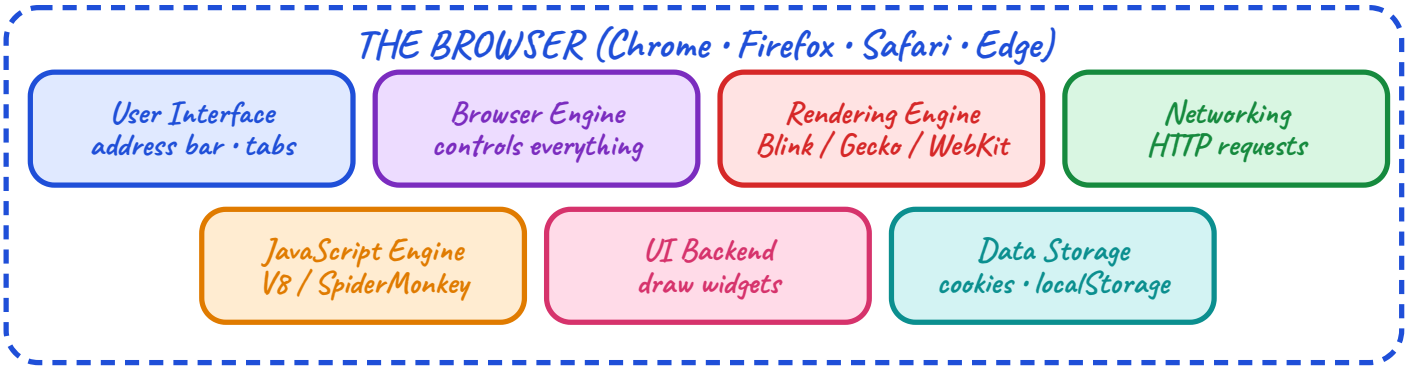
Human-friendly address (**asadokz18.com**). Bought yearly from a **registrar** (Namecheap, GoDaddy). Parts: name + TLD (.com .pk .org .dev).

Web hosting

Rented space on a server **where your files live** 24/7. Domain = address, hosting = the house.

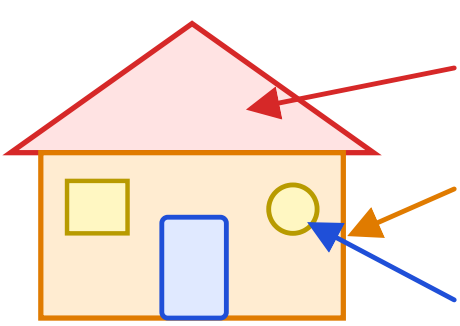
Domain (address) → **DNS** (finds IP) → **Hosting server** (stores your site) → **Visitor sees page**

Browser – the app that shows websites



Popular: **Chrome** (Blink/V8), **Firefox** (Gecko), **Safari** (WebKit), **Edge** (Blink). Always test on more than one browser (**cross-browser compatibility**).

4. The Three Building Blocks



In the browser

HTML → DOM

CSS → CSSOM

JS changes DOM → page updates

Language	Job	Example
HTML HyperText Markup Language	**Structure & content** – headings, paragraphs, images, links, forms, tables. Uses tags.	`<h1>Hello</h1>`
CSS Cascading Style Sheets	**Style & layout** – colours, fonts, spacing, positions, animations, responsive design.	`h1 { color: red; }`
JavaScript programming language	**Behaviour & logic** – clicks, forms, fetching data, animations, apps.	`alert("Hi");`

Working together – one tiny example

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>My Page</title>
  <link rel="stylesheet" href="style.css">      <!-- CSS: how it looks -->
</head>
<body>
  <h1 id="title">Hello!</h1>                    <!-- HTML: what it is -->
  <button id="btn">Click me</button>
  <script src="app.js" defer></script>          <!-- JS: what it does -->
</body>
</html>
```

style.css

```
#title { color: teal; font-size: 40px; }
button { padding: 10px 20px; }
```

app.js

```
document.querySelector("#btn").onclick = () => {
  document.querySelector("#title").textContent =
"Clicked!";
};
```

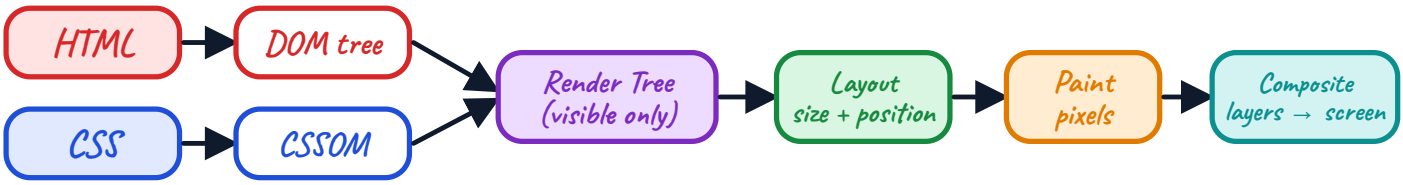
Learning order

① HTML → ② CSS → ③ JavaScript → ④ a framework (React) → ⑤ back-end (Node.js) → ⑥ database. Don't skip the basics!

Rule: separate concerns – keep HTML, CSS and JS in **separate files**. Other languages: TypeScript (typed JS), SASS (CSS tool), WebAssembly (fast code in browser).

5. Inside the Browser (Rendering)

How does the browser turn code into pixels? This is the **Critical Rendering Path**:



JavaScript can change the DOM / CSSOM at any time ➜ triggers re-layout & re-paint (expensive!)

1. **Parse HTML** → builds the **DOM** (Document Object Model) – a tree of all elements.
2. **Parse CSS** → builds the **CSSOM** (CSS Object Model) – all style rules.
3. DOM + CSSOM → **Render tree** (only visible elements; ***display:none*** is left out).
4. **Layout (reflow)** – calculate size and position of every box.
5. **Paint** – fill pixels: text, colours, borders, images, shadows.
6. **Composite** – layers are combined by the GPU and shown on screen.

Render-blocking

CSS blocks rendering (browser waits for styles). A plain **<script> blocks HTML parsing** → use ***defer*** / ***async*** or put scripts at the end of body.

Fast animations

Changing ***transform*** and ***opacity*** only needs **composite** – smooth. Changing width / top forces layout again – slow.

JavaScript engine & event loop

➜ JS runs in an engine (**V8** in Chrome/Node). It is **single-threaded**: one call stack. Async tasks (timers, fetch, events) wait in queues and run when the stack is empty (**event loop**).

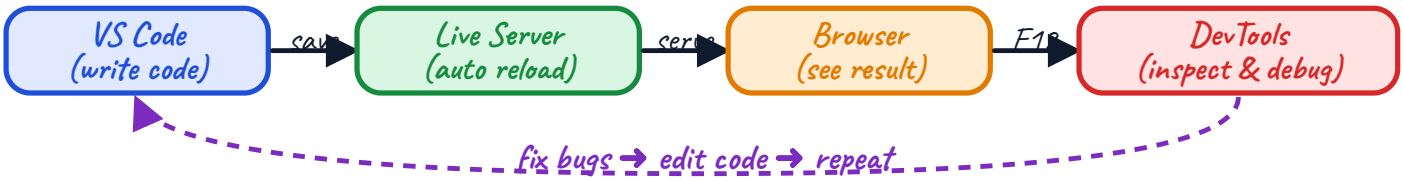
Browser storage & features

Feature	Use
Cache	Saves files (images, CSS, JS) to load pages faster next time
Cookies	Tiny data (≈4 KB) sent to the server with every request – login sessions
localStorage / sessionStorage	Up to ≈5 MB in the browser; permanent / until tab closes
IndexedDB	Large structured data for offline apps
Same-Origin Policu	A page can't read data of another origin (protocol + domain + port) – basic security rule

Try it

Press **F12** → **Performance** and **Network** tabs to see the rendering steps and loading order of any website.

6. Tools You Need



1) Code editor / IDE

- **VS Code** (free, most popular), Sublime Text, WebStorm, Notepad++. Features: syntax colours, auto-complete, extensions, built-in terminal, Git.
- Must-have VS Code extensions: **Live Server**, **Prettier** (formatter), **ESLint**, Auto Rename Tag, GitLens, Path Intellisense. Shortcuts: ***Ctrl+S*** save · ***Ctrl+/*** comment · ***Alt+***↑↓ move line · ***Ctrl+D*** multi-select · ***Ctrl+P*** open file · **!** + Tab → HTML skeleton (Emmet).

2) Browser DevTools (press F12 or Ctrl+Shift+I)

Tab	What you do
Elements	See / edit HTML & CSS live, box model, hover states, responsive mode (phone preview)
Console	Run JavaScript, see errors & ***console.log()*** output
Sources	Read JS files, set **breakpoints**, step through code
Network	Every request: URL, status code, size, time, headers, response (find slow files!)
Application	Cookies, localStorage, service workers, cache
Performance / Lighthouse	Speed, SEO, accessibility, best-practice report

3) Terminal (command line)

Common commands

```
pwd            # where am I?
ls  (dir)      # list files
cd folder      # go into folder
cd ..          # go back
mkdir app      # make folder
touch a.js     # make file
rm file        # delete file
code .         # open in VS Code
clear          # clean screen
```

Terminal is used for **Git**, **npm**, running servers (***node app.js***), deploying, Docker.

Windows: PowerShell / Git Bash / WSL · **Mac / Linux:** Terminal (bash / zsh).

Tip

Press **Tab** to auto-complete names, ↑ for previous command, **Ctrl+C** to stop a running program.

4) Live Server & other tools

- **Live Server** – VS Code extension: starts a local server and **reloads the browser automatically** when you save. Others: Vite dev server, ***npx serve***. Local address: ***http://localhost:5500*** (127.0.0.1).
- **Postman / Thunder Client** – test APIs · **Figma** – design · **Git & GitHub** – code history · **Chrome Lighthouse** · **caniuse.com** · **MDN Web Docs** (best documentation).

7. HTTP Basics – Part 1

→ HTTP (HyperText Transfer Protocol) = the set of rules browsers and servers use to talk. It is **stateless** (each request is independent) and works as request → response.

HTTP REQUEST (browser → server)

```
GET /products?id=5 HTTP/1.1      request line: METHOD path version
Host: shop.com
User-Agent: Chrome/120           headers (extra info)
Cookie: session=abc123
Accept: application/json

{ "name": "Asad" }               body (POST / PUT only)
```

HTTP RESPONSE (server → browser)

```
HTTP/1.1 200 OK                  status line: version code text
Content-Type: text/html
Cache-Control: max-age=3600      headers
Set-Cookie: theme=dark
Content-Length: 5120

<html> ... </html>               body (HTML / JSON / file)
```

HTTP methods (verbs)

Method	Purpose	CRUD	Body? / Safe?
GET	Read / fetch data (open a page, list users)	Read	No body · safe
POST	Create new data / submit form / login	Create	Has body
PUT	Replace a whole record	Update	Has body
PATCH	Change only some fields	Update	Has body
DELETE	Remove a record	Delete	Usually none
HEAD / OPTIONS	Only headers / ask which methods are allowed (CORS preflight)	–	–

GET vs POST: GET puts data in the URL (`?q=html&page=2`) – visible, bookmarkable, cached. POST sends data in the body – hidden, bigger, used for passwords & forms.

Status codes – the server's answer

Range	Meaning	Common codes
1xx	Information	100 Continue · 101 Switching Protocols (WebSocket)
2xx	Success ✔	**200** OK · **201** Created · **204** No Content
3xx	Redirect →	**301** Moved Permanently · **302** Found (temporary) · **304** Not Modified (use cache)
4xx	Client error ✘	**400** Bad Request · **401** Unauthorized (login needed) · **403** Forbidden · **404** Not Found · **429** Too Many Requests
5xx	Server error 💥	**500** Internal Server Error · **502** Bad Gateway · **503** Service Unavailable · **504** Timeout

Memory trick

2xx = "all good" · 3xx = "go somewhere else" · 4xx = "**you** made a mistake" · 5xx = "**we** (server) made a mistake".

7. HTTP Basics – Part 2 (Headers, HTTPS)

Important HTTP headers

Header	Meaning
Host	Which website (domain) the request is for
User-Agent	Which browser / device is asking
Accept / Content-Type	Format wanted / format being sent: **application/json**, **text/html**, **multipart/form-data**
Authorization	Credentials: **Bearer eyJhbGci...** (JWT) or API key
Cookie / Set-Cookie	Browser sends cookies / server asks browser to save one
Cache-Control / ETag	How long to cache · version tag for cache validation
Location	Where to go in a 3xx redirect
Access-Control-Allow-Origin	CORS – which other websites may call this server
Content-Security-Policy	Security rule: which scripts / styles are allowed (stops XSS)

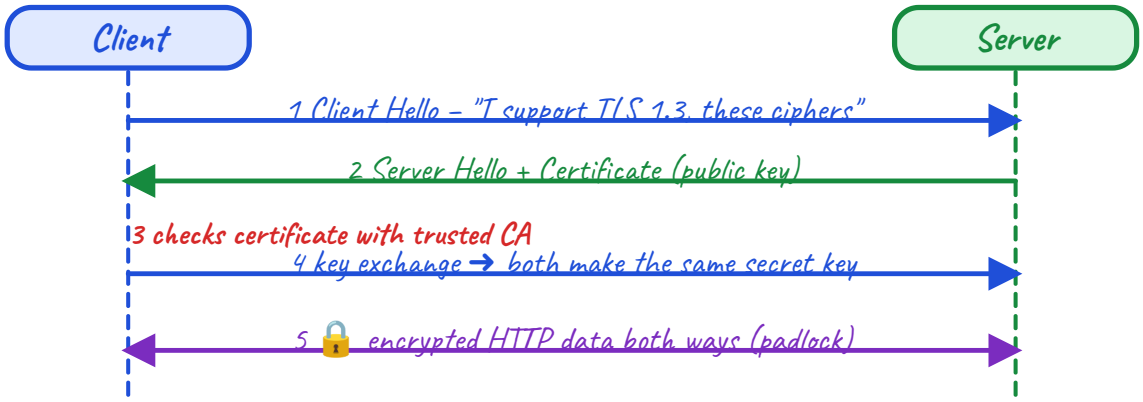
HTTP vs HTTPS

HTTP (port 80)

Data travels as **plain text**. Anyone on the network (Wi-Fi hacker) can read or change passwords. Browser shows "Not secure".

HTTPS (port 443) 🔒

HTTP + **TLS/SSL encryption**: **encrypted**, **authenticated** (real site) and **untampered**. Needs an SSL certificate (free: **Let's Encrypt**).



HTTP version	Features
HTTP/1.1 (1997)	Text-based, one request at a time per connection, keep-alive
HTTP/2 (2015)	Binary, **multiplexing** (many requests on one connection), header compression
HTTP/3 (2022)	Runs on **QUIC (UDP)** – faster, better on bad mobile networks

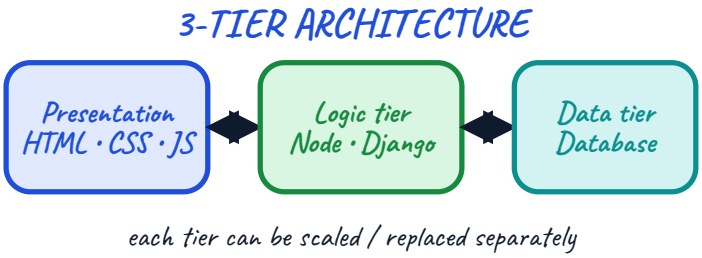
Cookies & caching in one view

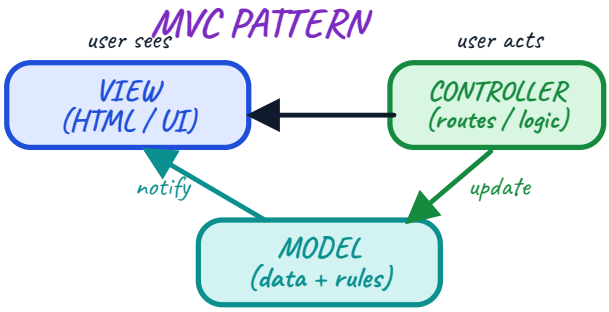
- **Cookie** – server sends **Set-Cookie**; browser sends it back on every request (login, preferences, tracking). Flags: **HttpOnly** (JS can't read), **Secure** (HTTPS only), **SameSite** (stops CSRF), **Max-Age**.
- **Caching** – browser / CDN keeps copies so the same file isn't downloaded again: **Cache-Control: max-age=31536000**; 304 Not Modified saves bandwidth.

Common ports

HTTP 80 · HTTPS 443 · SSH 22 · FTP 21 · SMTP 25 · PostgreSQL 5432 · MySQL 3306 · MongoDB 27017 · Node dev 3000.

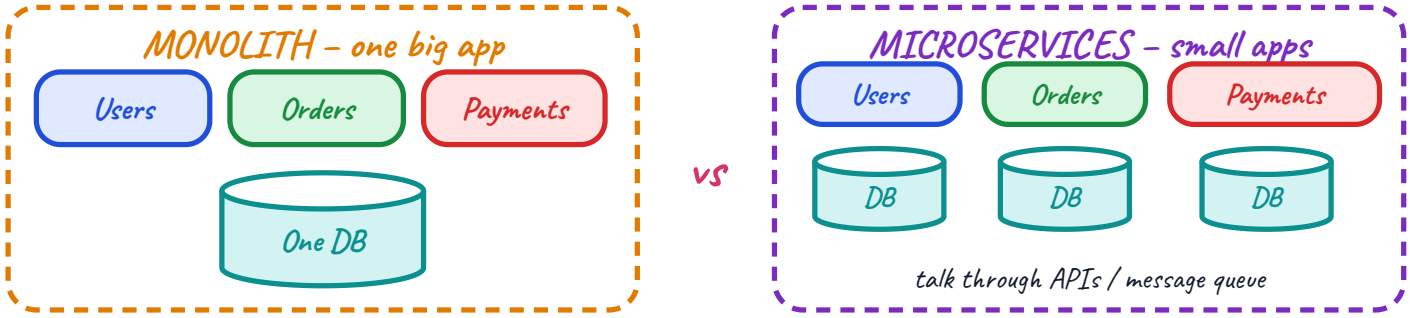
8. Web Architecture & Rendering Patterns





→ **3-tier:** Presentation (UI) ↔ Logic (server) ↔ Data (database). **MVC:** Model = data, View = what user sees, Controller = handles input & connects them (used in Laravel, Django, Rails, Express + EJS).

Monolith vs Microservices



	Monolith	Microservices
Good for	Startups, small teams, simple to build & deploy	Big systems, many teams, independent scaling
Problems	Hard to scale one part; one bug can crash all	Complex, network calls, harder debugging

Where is the page built? (Rendering patterns)

Pattern	How it works	Examples / tools
CSR – Client-Side Rendering	Server sends empty HTML + big JS; the **browser** builds the page. Fast after load, weaker SEO.	React / Vue SPA (Vite)
SSR – Server-Side Rendering	Server builds full HTML **per request**. Good SEO & first load.	Next.js, Nuxt, PHP, Django
SSG – Static Site Generation	HTML built **once at build time**, served from CDN. Fastest & cheapest.	Astro, Hugo, Gatsby, Next export
ISR / Hybrid	Static pages re-generated in the background.	Next.js ISR

→ **SPA** (Single Page App) – one HTML page, JS swaps content, no full reload (Gmail). **MPA** (Multi-Page App) – each link loads a new page (Wikipedia).

→ **JAMstack** – JavaScript + APIs + prebuilt Markup. **Serverless** – write functions, the cloud runs them on demand (AWS Lambda, Vercel Functions).

Scaling a website

Vertical
Bigger server (more RAM / CPU).

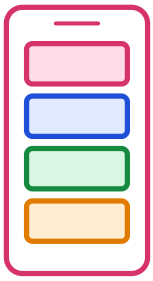
Horizontal
More servers + **load balancer**.

Caching
Redis / CDN reduce load.

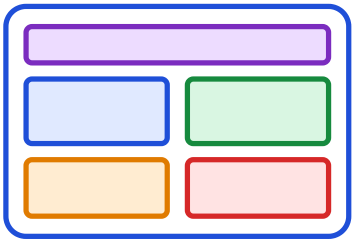
9. Front-end Basics

Responsive design

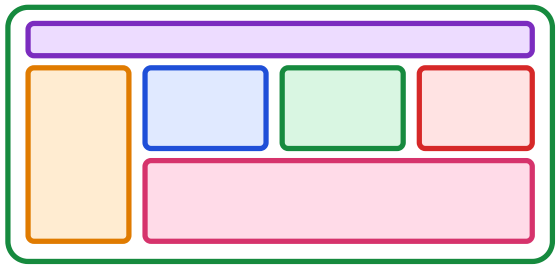
One website that adapts to **every screen size**. Google ranks mobile-friendly sites higher; over 60% of traffic is mobile.



Phone < 600px
1 column, menu ☰



Tablet 600–1024px
2 columns



Desktop > 1024px – sidebar + 3 columns

Responsive checklist

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">   <!-- 1. viewport -->
```

```
img { max-width: 100%; height: auto; }                    /* 2. flexible images          */
.container { width: min(100% - 2rem, 1100px); margin-inline: auto; }
.cards { display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); gap: 16px; }  /* 3. grid / flex */
@media (min-width: 768px) { .menu { display: flex; } }    /* 4. media queries (mobile-first) */
```

UI & UX

UI – User Interface

How it **looks**: colours, fonts, buttons, spacing, icons, layout. Tool: Figma.

UX – User Experience

How it **feels**: easy to use, fast, clear navigation, no confusion, helpful errors.

→ Good UX rules: consistency · clear call-to-action buttons · readable text (16px+, contrast) · fast loading · mobile first · feedback (loading spinner, success message) · less clicks.

Frameworks & libraries

Name	Type	Notes
React	JS library (Meta)	Components + JSX; most popular; Next.js on top
Vue	JS framework	Easy to learn; Nuxt on top
Angular	JS framework (Google)	Full framework with TypeScript; for big apps
Svelte / Astro	Compiler / framework	Very fast, little JS shipped
Tailwind / Bootstrap	CSS frameworks	Utility classes / ready components
jQuery / Alpine.js	Small helpers	Old-school DOM helper / light interactivity

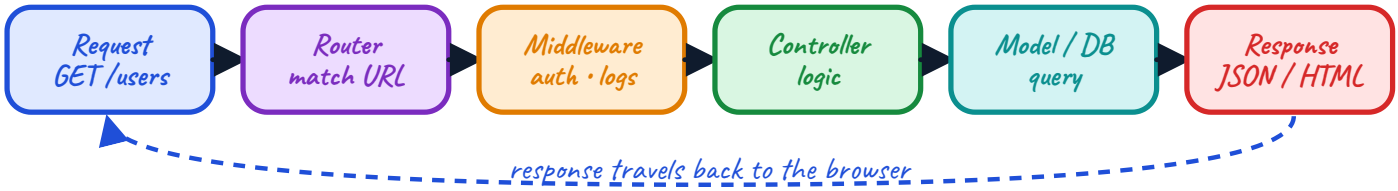
→ **Library** = you call it (a toolbox). **Framework** = it calls your code (rules & structure). Learn **JavaScript well first**, then a framework. Concepts: components, props, state, hooks, routing, state management (Redux, Zustand), forms.

Accessibility (a11y)

→ Making sites usable by **everyone** (blind, deaf, motor / cognitive disabilities, keyboard-only). Use **semantic HTML** (`<nav> <main> <button>`), **alt** text, labels on inputs, colour contrast 4.5:1, keyboard navigation, focus outlines, ARIA only when needed, captions for videos. Check with **Lighthouse** / **axe**. Follows **WCAG**.

10. Back-end Basics

What does the server do?



→ **Receives** requests · **routes** them to the right code · **validates** input · checks **login / permissions** · applies **business logic** · reads / writes the **database** · sends **HTML / JSON / files** back · sends emails · handles payments · writes **logs**.

Back-end languages & frameworks

Language	Frameworks	Known for
JavaScript (Node.js)	Express, NestJS, Fastify	Same language front & back; real-time apps
Python	Django, Flask, FastAPI	Easy; AI / data; fast development
PHP	Laravel, WordPress	Powers ~40% of websites; cheap hosting
Java	Spring Boot	Big companies, banks, Android
C# (.NET)	ASP.NET Core	Microsoft, enterprise
Go / Rust / Ruby	Gin / Actix / Rails	Speed & concurrency / safety / developer joy

Simple Node.js server

server.js (run: node server.js)

```
const http = require("http");
http.createServer((req, res) => {
  if (req.url === "/") {
    res.writeHead(200, { "Content-Type": "text/html" });
    res.end("<h1>Hello from the server!</h1>");
  } else {
    res.writeHead(404); res.end("Not found");
  }
}).listen(3000, () => console.log("http://localhost:3000"));
```

Same thing with Express (easier)

app.js (npm i express)

```
const express = require("express");
const app = express();
app.use(express.json());                          // middleware: read JSON bodies
app.get("/", (req, res) => res.send("Hello!"));
app.get("/api/users/:id", (req, res) => res.json({ id: req.params.id }));
app.post("/api/users", (req, res) => res.status(201).json(req.body));
app.listen(3000);
```

Web servers

Nginx / Apache – serve static files, reverse proxy, HTTPS, load balancing in front of your app.

Good back-end habits

Validate input · hide secrets in **.env** · log errors · use proper status codes · rate limit.

11. Authentication & Sessions

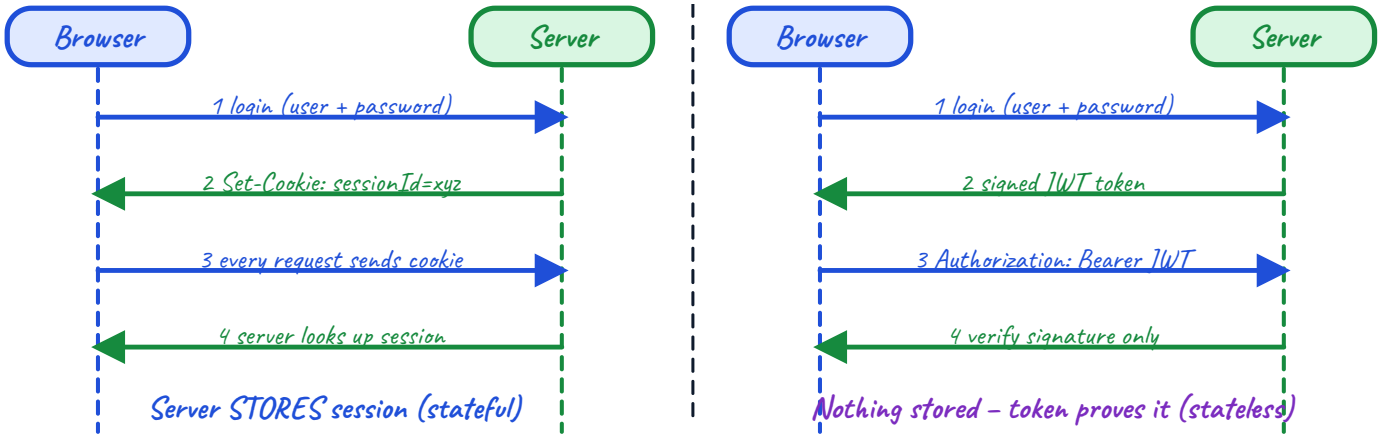
Authentication (AuthN)

Who are you? Proving identity – username + password, OTP, fingerprint.

Authorization (AuthZ)

What can you do? Permissions – admin vs normal user (roles / RBAC).

Session vs Token (JWT)



	Session + Cookie	JWT (JSON Web Token)
Stored	On server (memory / Redis / DB); browser keeps only the id	In browser (cookie / memory); server stores nothing
Logout	Easy – delete session	Harder – wait for expiry / blacklist
Best for	Normal websites	APIs, mobile apps, microservices

JWT structure: *header*.*payload*.*signature* (3 parts, Base64). Payload = user id, role, expiry. Never put secrets in it – it is only **signed**, not encrypted.

Login flow (step by step)

1. User sends email + password (over **HTTPS**, via POST). 2. Server finds user, **compares hashed password** (bcrypt). 3. If OK → create session / JWT. 4. Browser stores cookie (**HttpOnly, Secure, SameSite**). 5. Each next request carries it. 6. Server checks it + role, then answers. 7. Logout → delete session / cookie.

```
const hash = await bcrypt.hash(password, 10);          // store ONLY the hash
const ok = await bcrypt.compare(input, user.hash);     // login check
const token = jwt.sign({ id: user.id, role: "user" }, process.env.JWT_SECRET, { expiresIn: "1h" });
const data = jwt.verify(token, process.env.JWT_SECRET);   // throws if fake / expired
```

More authentication topics

- **OAuth 2.0 / "Login with Google"** – let another provider verify the user; you get a token (never the password). **SSO** – one login for many apps. **OpenID Connect** = identity on top of OAuth.
- **2FA / MFA** – password + a second proof (SMS / authenticator app). **Passkeys** – passwordless login with biometrics. **API keys** – for machines. **Refresh tokens** – short access token + long refresh token.

Never

store plain-text passwords · put tokens in URLs · trust the front-end for security (always check on server) · use weak hashing (MD5, SHA1).

12. Databases

What is a database?

An organised collection of data stored on a server, managed by a DBMS (Database Management System), so that apps can **store, find, update and delete** data fast and safely (users, products, orders, posts).

SQL (relational) databases

users

id (PK)	name	email
1	Asad	a@x.com
2	Ali	ali@x.com
3	Sara	s@x.com

1 : N

posts

id	user_id (FK)	title
10	1	Intro
11	1	HTML
12	3	CSS

Key words

- Table = rows (records) + columns
- PK primary key = unique id
- FK foreign key = link to another table
- 1:N one user, many posts
- Relation: 1:1 · 1:N · N:M

SQL (Structured Query Language)

```
CREATE TABLE users (id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(50), email VARCHAR(100) UNIQUE);
INSERT INTO users (name, email) VALUES ('Asad', 'a@x.com');            -- Create
SELECT name, email FROM users WHERE id = 1 ORDER BY name LIMIT 10;      -- Read
UPDATE users SET name = 'Asad Ullah' WHERE id = 1;                      -- Update
DELETE FROM users WHERE id = 1;                                         -- Delete
SELECT u.name, p.title FROM users u JOIN posts p ON p.user_id = u.id;   -- combine tables (JOIN)
```

Examples: MySQL, PostgreSQL, SQLite, SQL Server, Oracle. Strict **schema**, relations, **ACID** transactions (Atomic, Consistent, Isolated, Durable) – ideal for money & orders.

NoSQL databases

Type	Stores	Examples
Document	JSON-like documents (flexible fields)	**MongoDB**, Firestore
Key-Value	Fast key → value (cache, sessions)	**Redis**, DynamoDB
Column	Huge write-heavy data	Cassandra
Graph	Relationships (social networks)	Neo4j

MongoDB document

```
{ "_id": "64f1", "name": "Asad", "skills": ["HTML", "CSS", "JS"], "address": { "city": "Lahore" } }
```

CRUD ↔ HTTP ↔ SQL

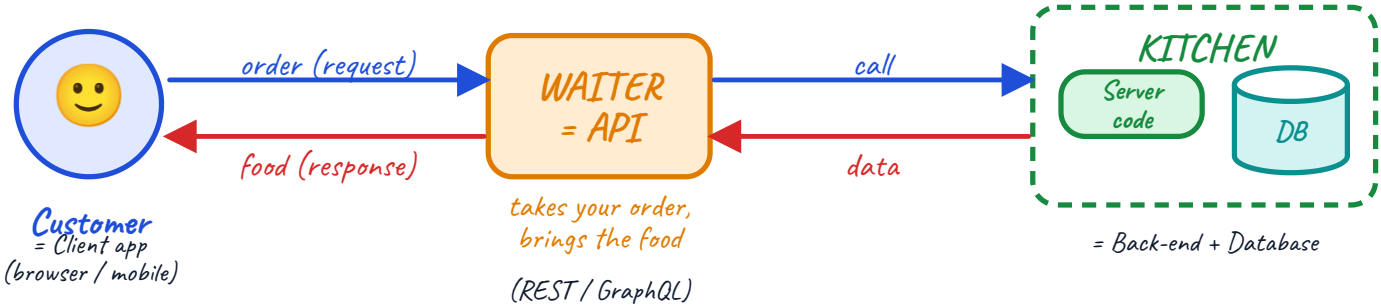
CRUD	HTTP method	SQL	REST example
Create	POST	INSERT	POST /users
Read	GET	SELECT	GET /users/5
Update	PUT / PATCH	UPDATE	PATCH /users/5
Delete	DELETE	DELETE	DELETE /users/5

More: Index = speeds up searches · **ORM** (Prisma, Sequelize, Mongoose) = use objects instead of SQL · **Normalization** = no duplicate data · **Backups** & replication · **SQL vs NoSQL:** structured & relations → SQL; flexible & huge scale → NoSQL.

13. APIs & JSON

What is an API?

API = Application Programming Interface – a contract that lets one program talk to another. Like a waiter: you don't enter the kitchen; you order from the menu.



REST API

REST = Representational State Transfer. Uses **URLs for things (nouns)** and **HTTP methods for actions (verbs)**. Stateless. Returns JSON.

Method	Endpoint	Action
GET	/api/users	List all users
GET	/api/users/5	Get user 5 (query: ?page=2&limit=10)
POST	/api/users	Create a user (data in body) → 201
PUT/PATCH	/api/users/5	Update user 5
DELETE	/api/users/5	Delete user 5 → 204

JSON – JavaScript Object Notation

```
{ "id": 5, "name": "Asad", "isStudent": true, "zip": null,
  "skills": ["HTML", "CSS", "JS"],
  "address": { "city": "Lahore" } }
```

→ Text format for data. Types: string, number, boolean, null, array, object. Keys in **"double quotes"**; no comments, no trailing commas. **JSON.stringify(obj) ↔ JSON.parse(text)**. Send data: **fetch(url, {method:"POST", headers, body: JSON.stringify(obj)})**. Others: XML, YAML, CSV.

Fetch data (front-end calls the API)

```
async function loadUsers() {
  try {
    const res = await fetch("https://jsonplaceholder.typicode.com/users");
    if (!res.ok) throw new Error("HTTP " + res.status);      // fetch doesn't throw on 404 / 500!
    const users = await res.json();
    document.querySelector("#list").innerHTML = users.map(u => `<li>${u.name}</li>`).join("");
  } catch (err) { console.error(err); }
}
```

API types

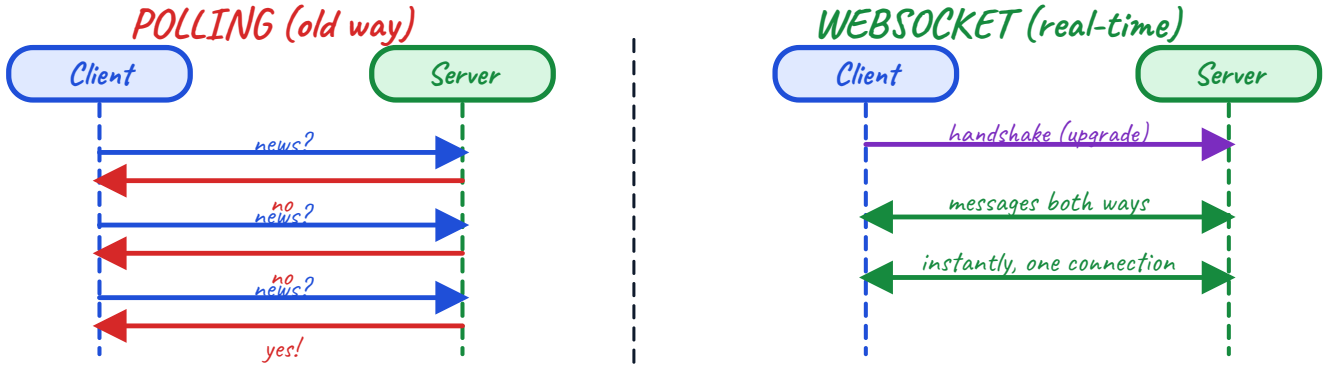
REST (most common) · **GraphQL** (ask exactly the fields you need, one endpoint) · **gRPC** · **SOAP** (old XML) · **WebSocket**.

API good practice

API keys / tokens · **rate limiting** · versioning **/v1/** · CORS · docs (Swagger / OpenAPI, Postman) · consistent errors.

14. Real-time Web, PWA & More

HTTP normally works as: client asks → server answers. For chat, live scores, notifications and multiplayer games we need **real-time** updates.



Technique	How	Use
Polling	Client asks again and again every few seconds	Simple, wasteful
Long polling	Server holds the request until it has news	Older chat systems
SSE (Server-Sent Events)	One-way stream **server → client** over HTTP	Live feeds, notifications
WebSocket	One **two-way** connection that stays open (**ws://**, **wss://**)	Chat, games, trading, collaboration
WebRTC	Direct browser-to-browser audio / video / data	Video calls (Meet, Zoom web)

WebSocket client + Socket.io server

```
const ws = new WebSocket("wss://example.com/chat");
ws.onopen = () => ws.send("Hello!");
ws.onmessage = (e) => console.log("New message:", e.data);
// Node: const io = require("socket.io")(server);  io.on("connection", s => s.on("msg", m => io.emit("msg", m)));
```

PWA – Progressive Web App

A website that behaves like a **mobile app**: installable, works **offline**, fast, sends push notifications. Needs **HTTPS** + 3 things:

1 Manifest
manifest.json: name, icons, colours, start_url, display: standalone.

2 Service Worker
JS that runs in background; caches files, handles offline, push, background sync.

3 HTTPS
Required for service workers.

service worker (sw.js)

```
navigator.serviceWorker.register("/sw.js");                                    // in page
self.addEventListener("fetch", e => e.respondWith(caches.match(e.request).then(r => r || fetch(e.request))));
```

Other modern web technologies

- **WebAssembly (Wasm)** – near-native speed code (C++ / Rust) in the browser (games, Figma). • **Web Components** – custom reusable HTML elements. • **Web Workers** – background threads. • **Canvas / WebGL / Three.js** – 2D / 3D graphics. • **IndexedDB** – offline database. • **Geolocation, Camera, Notifications, Clipboard** APIs. • **Edge computing** – run code near the user (Cloudflare Workers).

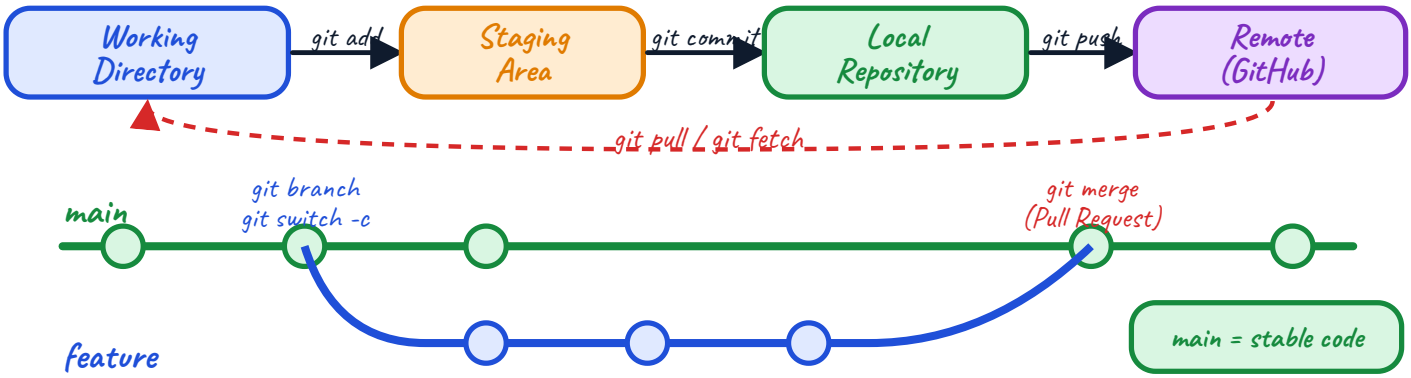
15. Git & GitHub

Version control

A system that **records every change** of your code, so you can go back, compare, work in a team and never lose work ("undo for the whole project").

Git vs GitHub

Git = the tool on your computer (local). **GitHub** = website to host Git repositories online & collaborate (also GitLab, Bitbucket).



Main commands

Setup & first commit

```
git config --global user.name "asadokz18"
git config --global user.email "you@mail.com"
git init                        # start a repo
git clone <url>                 # copy a repo
git status                      # what changed?
git add .                       # stage all files
git commit -m "Add navbar"      # save snapshot
git log --oneline               # history
```

Branch, remote & undo

```
git branch                      # list branches
git switch -c feature/login     # new branch
git switch main                 # change branch
git merge feature/login         # join into main
git remote add origin <url>
git push -u origin main         # upload
git pull                        # download + merge
git restore file.js             # discard changes
git revert HEAD                 # undo a commit safely
```

Important ideas

- **Commit** = a saved snapshot with a message (write clear ones: *"Fix login bug"*). **Branch** = a separate line of work (main stays stable). **Merge** = combine branches; **merge conflict** = same lines changed in both → fix manually, then add + commit.
- ***.gitignore*** – files Git must ignore: ***node_modules/***, ***.env***, ***dist/***, ***.DS_Store***. ***README.md*** – project description. ***HEAD*** = where you are now. ***git stash*** = hide unfinished work. ***git rebase*** = replay commits (advanced).

GitHub features

Feature	What it is
Repository	Project folder online (public / private)
Fork	Your own copy of someone's repository
Pull Request (PR)	"Please merge my branch" – code review before merging
Issues / Projects	Bug reports, tasks, Kanban board
Actions	Automation: tests + deploy on every push (CI/CD)
GitHub Pages	Free hosting of static sites (***username.github.io***)

Workflow

Fork / clone → new branch → code → commit often → push → open Pull Request → review → merge → delete branch. Build a **GitHub profile** with your projects – it is your developer CV!

16. Package Managers & Build Tools

npm – Node Package Manager

→ Downloads and manages **packages** (ready-made code libraries: React, Express, Lodash, Axios, Tailwind…) from the npm registry. Comes with **Node.js**. Alternatives: **yarn**, **pnpm**, **bun**.

Command	Does
npm init -y	Create **package.json**
npm install axios	Install package (saved in dependencies) → **node_modules/**
npm i -D vite	Dev dependency (only for development)
npm install	Install everything listed in package.json
npm run dev / build	Run a script from package.json

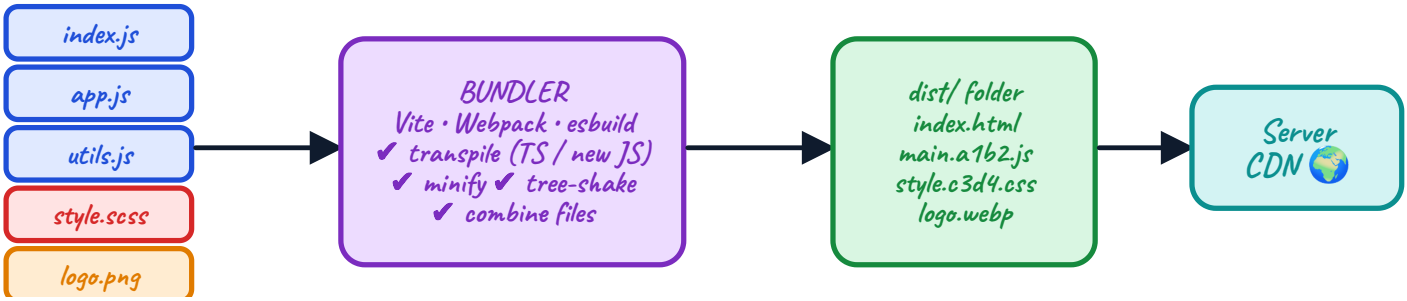
package.json – the project's ID card

package.json

```
{
  "name": "my-app",
  "version": "1.0.0",
  "scripts": { "dev": "vite", "build": "vite build", "start": "node server.js", "test": "jest" },
  "dependencies": { "express": "^4.19.2", "axios": "^1.7.0" },
  "devDependencies": { "vite": "^5.0.0", "eslint": "^9.0.0" }
}
```

→ **package-lock.json** = exact versions (commit it) · **node_modules/** = downloaded code (never commit; huge) · Versions MAJOR.MINOR.PATCH: **^1.2.3** = minor + patch updates ok, **~1.2.3** = patch only.

Bundlers & build tools



Tools: Vite / Webpack / Parcel / esbuild bundle files & give a dev server with **hot reload** · **Babel / TypeScript** transpile modern code · **Sass / PostCSS / Tailwind** for CSS · **ESLint / Prettier** check & format.

Build optimisations: minify (remove spaces, shorten names) · **tree-shaking** (drop unused code) · **code splitting** (load pages on demand) · **hashed file names** for cache busting.

Environment variables (.env)

.env (never commit this file!)

```
PORT=3000
JWT_SECRET=super-secret-key        # + DATABASE_URL, API keys...
```

```
require("dotenv").config();          // Node: load .env      (Vite: import.meta.env.VITE_API_URL)
const port = process.env.PORT || 3000;
```

Why env variables?

Keep **secrets** (passwords, API keys) out of code & GitHub, and use different settings for **development / staging / production**.

17. Web Security Basics

Rule #1: never trust user input and never trust the front-end – always validate and check permissions on the server.

1) Use HTTPS 🔒

Encrypts data between browser and server (TLS). Free certificates from **Let's Encrypt**. Redirect HTTP → HTTPS, add header ***Strict-Transport-Security (HSTS)***, mark cookies **Secure**.

2) XSS – Cross-Site Scripting



XSS = Cross-Site Scripting: attacker's JavaScript runs in the victim's browser on YOUR site

Prevent XSS

```
el.innerHTML = userInput;       // ✘ DANGEROUS – runs scripts
el.textContent = userInput;     // ✔ safe – treated as plain text
// Also: escape output · sanitise HTML (DOMPurify) · Content-Security-Policy header · HttpOnly cookies
```

3) SQL Injection

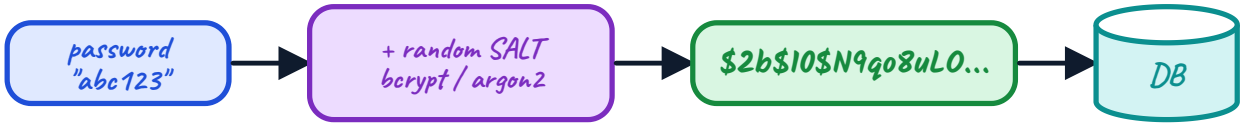
Attacker types SQL code in a form field to read / delete the database.

Vulnerable vs safe

```
-- ✘ DANGEROUS: string glued together. Input:  ' OR '1'='1' --
"SELECT * FROM users WHERE email = '" + email + "' AND password = '" + pass + "'"
--   becomes:  ... WHERE email = '' OR '1'='1' --'   → logs in as anyone!

-- ✔ SAFE: parameterised query (prepared statement)
db.query("SELECT * FROM users WHERE email = ? AND password_hash = ?", [email, hash]);
```

4) Passwords



Only the HASH is saved – one-way, can't be turned back into the password. Login: hash the typed password & compare.

5) Other important attacks & defences

Threat	What it is	Defence
CSRF	Evil site makes your browser send a request to a site you're logged in to	CSRF tokens, **SameSite** cookies
CORS	Browser blocks cross-origin API calls	Allow only trusted origins on the server
Brute force / DDoS	Guessing passwords / flooding the server	Rate limiting, lockout, CAPTCHA, CDN / WAF
Supply chain	Malicious / outdated npm packages	***npm audit***, lock versions, update

→ **OWASP Top 10** = list of most common web risks (broken access control, injection, misconfiguration…). Also: validate input, hide error details, keep secrets in **.env**, least privilege, update dependencies, backups, security headers, 2FA.

18. Performance & SEO

Users leave if a page takes more than 3 seconds. Fast sites = more visitors, sales and better Google ranking.

Speed tips

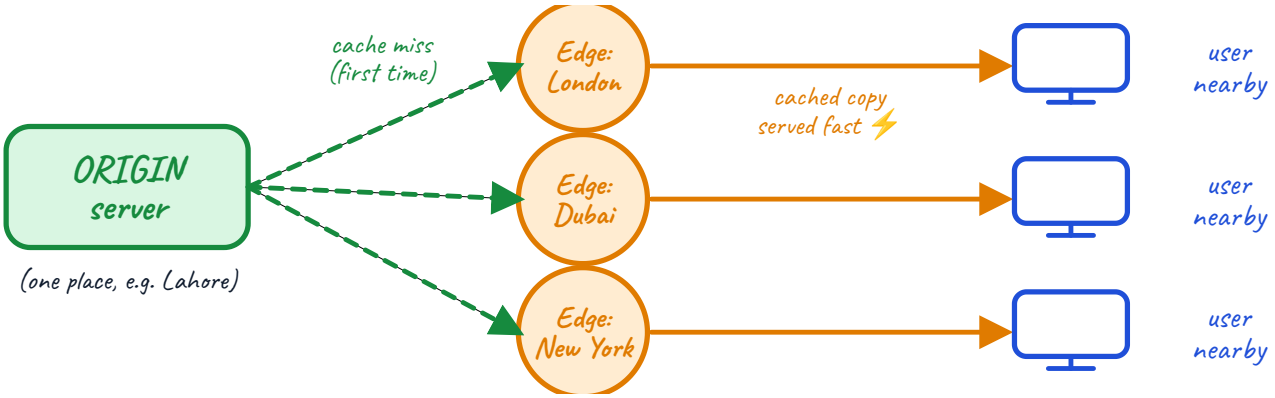
- → **Compress & resize images**; use **WebP / AVIF**; ***loading="lazy"***
- → **Minify** CSS / JS / HTML; bundle & split code
- → **Gzip / Brotli** compression on server
- → Use a **CDN** and **caching**
- → Fewer HTTP requests; HTTP/2 or 3
- → Load scripts with ***defer*** / ***async***
- → **Preload** key fonts (***rel="preload"***), ***font-display: swap***
- → Avoid heavy libraries; remove unused CSS
- → Set image width & height (no layout jump)
- → Fast server & **database indexes**; use SSR / SSG

Core Web Vitals (Google's user-experience metrics)

Metric	Measures	Good ✔	Poor ✘
LCP – Largest Contentful Paint	How fast the **main content** (big image / heading) appears	≤ 2.5 s	> 4 s
INP – Interaction to Next Paint	How fast the page **reacts** to clicks / taps (replaced FID)	≤ 200 ms	> 500 ms
CLS – Cumulative Layout Shift	How much things **jump around** while loading	≤ 0.1	> 0.25

Others: **TTFB** (time to first byte), **FCP** (first paint). Measure with **Lighthouse**, PageSpeed Insights, WebPageTest, Chrome DevTools Performance.

Caching & CDN



- → **CDN** (Content Delivery Network: Cloudflare, Akamai, CloudFront) copies your static files to servers around the world; users get them from the **nearest** one. Cache layers: browser cache → CDN → server cache (Redis) → database.

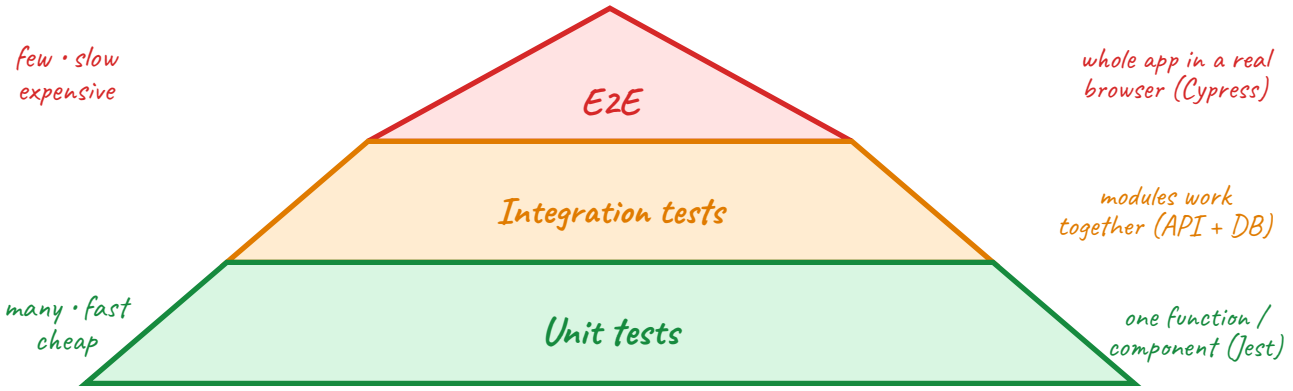
SEO – Search Engine Optimisation

- → **Title** (≈60 chars) & **meta description** (≈150)
- → One **h1**, logical headings, **semantic HTML**
- → Keywords in content, URLs & image **alt**
- → Mobile-friendly, HTTPS, fast
- → **sitemap.xml** & **robots.txt**; canonical URL
- → Quality content + **backlinks**
- → Open Graph / Twitter cards, **structured data** (schema.org JSON-LD)
- → Google Search Console & Analytics

19. Testing & Debugging

Why test?

Find bugs **before users do**, change code without fear, document how code should behave. "Code without tests is broken by design."



Test type	Checks	Tools
Unit test	One function / component alone	Jest, Vitest, Mocha
Integration test	Several parts together (route + database)	Supertest, Testing Library
End-to-End (E2E)	Whole user flow in a real browser (login → buy)	Cypress, Playwright, Selenium
Other	Manual QA · regression · performance / load · accessibility · cross-browser · security · UAT (user acceptance)	Lighthouse, JMeter, k6, BrowserStack

Jest unit test

```
function add(a, b) { return a + b; }
test("adds 2 + 3 to equal 5", () => {
  expect(add(2, 3)).toBe(5);
});
// run:  npm test      TDD = Test-Driven Development: write the failing test first, then the code
```

Debugging – find and fix bugs

1. **Reproduce** the bug (same steps every time). 2. **Read the error message** and the line number in the Console. 3. **Isolate** – comment out code / minimal example. 4. **Inspect values** with ***console.log()***, ***console.table()***, breakpoints, ***debugger;***. 5. Form a guess, **fix**, test again. 6. Add a test so it never returns.

Where	How
Browser DevTools	Console errors · Sources breakpoints / step over / watch · Network (status, payload) · Elements (CSS)
Node.js back-end	***node --inspect***, VS Code debugger, logs (winston / pino), Postman for APIs
Production	Error tracking (**Sentry**), monitoring, logs, analytics

Common bugs

Typos · undefined / null · wrong file path · forgot to save · CORS error · 404 / 500 · async timing · cache (hard refresh **Ctrl+Shift+R**).

Code quality

Linting (ESLint), **formatting** (Prettier), **code review**, small functions, meaningful names, comments for "why".

20. Deployment & Hosting

Deploy = put your website on a **server connected to the Internet** so everyone can open it with a URL.

Steps to deploy a site

1. Finish & test locally. 2. Push code to **GitHub**. 3. **Build** (***npm run build***) → ***dist/***. 4. Choose a **host**. 5. Connect repo / upload files & set **environment variables**. 6. Add **custom domain** + HTTPS. 7. Test live, set up **monitoring**.

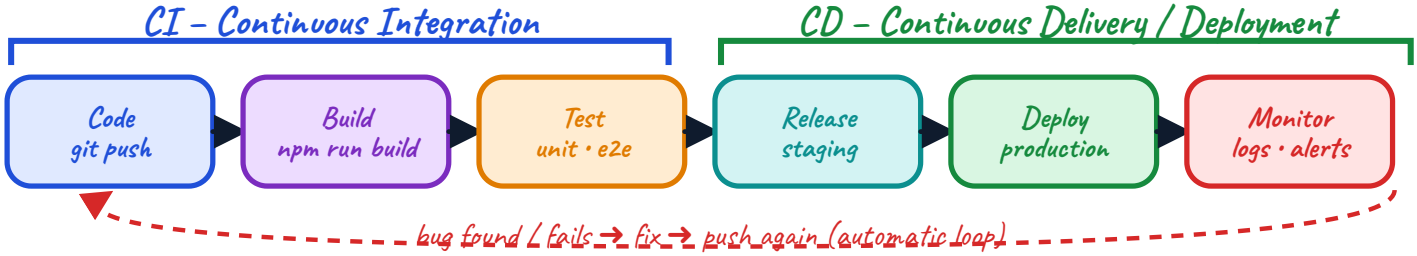
Hosting choices

Type	Examples	Best for
Static hosting (free)	GitHub Pages, Netlify, Vercel, Cloudflare Pages	HTML / CSS / JS, React builds, portfolios
PaaS (Platform as a Service)	Render, Railway, Heroku, Fly.io, Vercel (SSR)	Node / Python apps with DB – easy deploy
Shared hosting	Hostinger, Bluehost, cPanel hosts	WordPress / PHP, cheap, beginners
VPS	DigitalOcean, Linode, Hetzner, Contabo	Full control (Linux, Nginx), you manage everything
Cloud / IaaS	AWS, Google Cloud, Microsoft Azure	Big scalable apps, many services
Serverless / Edge	AWS Lambda, Cloudflare Workers	Pay per use, auto scale
Backend DB hosting	MongoDB Atlas, Supabase, Firebase, PlanetScale	Managed databases & auth

Connect a domain name

1. Buy domain at a registrar (Namecheap, GoDaddy, Cloudflare). 2. In **DNS settings** add **A** record → host IP or ***CNAME*** → ***myapp.vercel.app***. 3. Wait for DNS propagation (minutes – 48 h). 4. Enable **HTTPS** (auto SSL). 5. Redirect www ↔ non-www.

CI/CD – automate everything



CI = Continuous Integration

Developers merge code often; every push is **automatically built and tested** → bugs caught early.

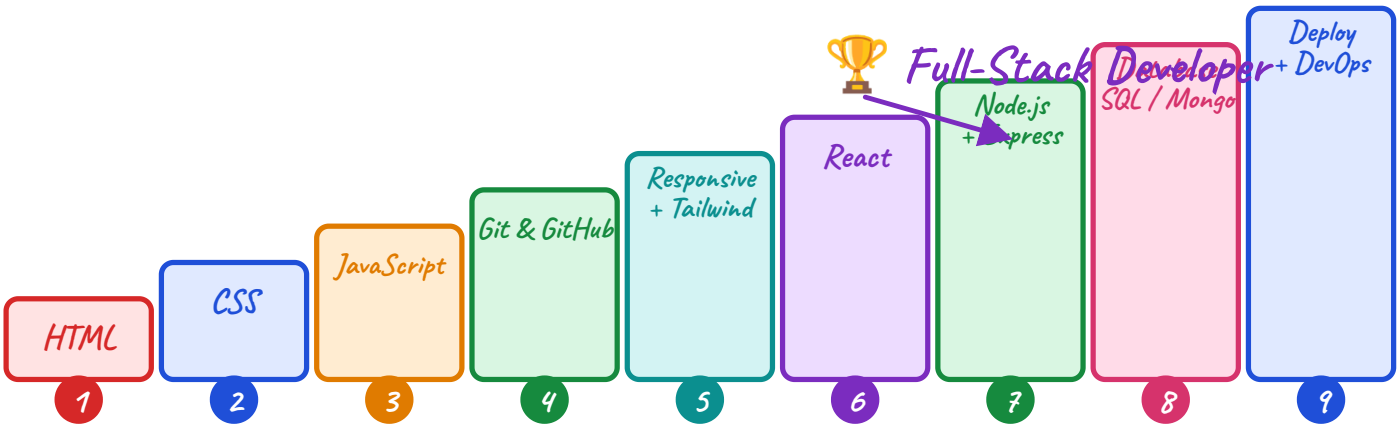
CD = Continuous Delivery / Deployment

Delivery: tested code is always ready to release (manual click). **Deployment:** goes live **automatically**.

Tools: GitHub Actions, GitLab CI, Jenkins, CircleCI. **Docker** = packs app + dependencies in a **container** ("works on my machine" fixed). **Environments:** development → staging (test copy) → production (live). Also: rollback, logs, uptime monitoring, backups, scaling, zero-downtime deploys.

21. Roadmap & Key Takeaways

Learning roadmap – step by step



Stage	Learn	Time*
1 Basics	How the web works, HTML (semantic, forms), CSS (box model, Flexbox, Grid, responsive)	1–2 months
2 JavaScript	Variables, functions, arrays, objects, DOM, events, async, fetch, ES6+	2–3 months
3 Tools	Terminal, Git & GitHub, npm, DevTools, Tailwind / Bootstrap	1 month
4 Front-end framework	React (components, hooks, router, state) + TypeScript basics	2–3 months
5 Back-end	Node.js + Express (or Django), REST APIs, auth (JWT), validation	2 months
6 Database	SQL (PostgreSQL) and / or MongoDB, ORM	1–2 months
7 Deploy & extras	Hosting, domains, CI/CD, Docker basics, testing, security, performance	1–2 months

*Rough time for 2–3 hours of daily practice. Consistency beats speed.

Project ideas (build these!)

Beginner

Personal portfolio · tribute page · landing page · calculator · to-do list · product cards

Intermediate

Weather app (API) · quiz game · movie search · blog · expense tracker · e-commerce UI

Advanced

Full-stack e-commerce · chat app (WebSocket) · social media clone · Kanban board · admin dashboard

Good habits ✔

- → Code **every day**, even 1 hour
- → Build **projects**, not only tutorials
- → Read **MDN** & official docs
- → Use **Git** from day one
- → Google errors & read Stack Overflow
- → Write clean, commented, formatted code
- → Learn to **debug** (DevTools)
- → Test on mobile & many browsers
- → Share work on GitHub / LinkedIn
- → Never stop learning – tech changes fast

Remember

HTML = structure · **CSS** = style · **JS** = behaviour · **Server** = logic · **Database** = memory · **API** = bridge · **Git** = safety net · **HTTPS** = security · **Deploy** = go live. Master the **fundamentals** – frameworks come and go!

22. Web Terms Glossary (A – Z)

Term	Meaning
API	Interface letting programs talk to each other
Back-end	Server side: logic, database, APIs
Bandwidth	Amount of data that can be transferred per second
Breakpoint	Screen width where the layout changes
Bug	Mistake / error in code
Cache	Saved copy of data for faster loading
CDN	Network of servers delivering files from the nearest location
CMS	Content Management System: WordPress, Strapi, Shopify
Cookie	Small data stored in the browser by a site
CORS	Rule controlling cross-origin requests
CRUD	Create, Read, Update, Delete
DNS	Converts domain names to IP addresses
DOM	Tree of the page's elements that JS can change
Endpoint	A URL of an API that does one job (/api/users)
Framework	Ready structure / rules for building apps (React, Django)
Front-end	What the user sees: HTML, CSS, JS
Hosting	Server space where a site lives
HTTP / HTTPS	Protocol for web communication / its encrypted version
IDE	Integrated Development Environment (VS Code, WebStorm)
IP address	Number identifying a device on a network
JSON	Lightweight text format for data
JWT	Signed token used for authentication
Latency	Delay before data starts to transfer (ping)
Middleware	Code that runs between request and response (auth, logging)
MVC	Model – View – Controller pattern
ORM	Use a database with objects instead of raw SQL
Repository (repo)	Project folder tracked by Git
REST	API style using URLs + HTTP methods
Route	URL path mapped to code
Scalability	Ability to handle more users / load
SEO	Search Engine Optimisation
SPA	Single Page Application
SSL / TLS	Encryption technology behind HTTPS
Stack	Set of technologies used: MERN = MongoDB, Express, React, Node
URL / URI	Web address of a resource
Viewport	Visible area of the page in the browser
Webhook	Server automatically calls your URL when an event happens
WebSocket	Two-way real-time connection
XSS	Attack that injects malicious scripts into a page

23. Interview Q&A – Part 1

Q1. What is the difference between front-end and back-end?

Front-end is the client side that users see (HTML, CSS, JS in the browser). Back-end is the server side: logic, database and APIs. Full-stack does both.

Q2. What happens when you type a URL and press Enter?

Browser checks cache → DNS finds the IP → TCP + TLS connection → HTTP request → server processes (maybe DB) → HTTP response with HTML → browser builds DOM + CSSOM → loads CSS/JS/images → renders the page.

Q3. What is the difference between HTTP and HTTPS?

HTTPS is HTTP encrypted with TLS: data is private, authenticated and cannot be changed on the way (port 443 vs 80).

Q4. Explain GET vs POST. What are PUT and PATCH?

GET reads data (in URL, cacheable); POST sends / creates data (in body). PUT replaces the whole resource; PATCH updates part of it.

Q5. What do 200, 301, 404 and 500 mean?

OK; moved permanently; not found; internal server error.

Q6. What is DNS?

The system that translates domain names into IP addresses.

Q7. What is an API? What is REST?

API lets programs communicate. REST is an API style using URLs for resources and HTTP methods for actions, usually returning JSON.

Q8. SQL vs NoSQL?

SQL: tables, fixed schema, relations, ACID (MySQL, PostgreSQL). NoSQL: flexible documents / key-value, easy horizontal scaling (MongoDB, Redis).

Q9. Session vs JWT authentication?

Session: server stores session, browser keeps a cookie id. JWT: signed token stored on the client; server stores nothing (stateless).

Q10. What is CORS?

Browser security rule that blocks web pages from calling other origins unless the server allows it with Access-Control-Allow-Origin headers.

23. Interview Q&A – Part 2

Q11. What is responsive design?

Layouts that adapt to all screen sizes using the viewport meta tag, flexible units, Flexbox / Grid and media queries.

Q12. CSR vs SSR vs SSG?

CSR builds the page in the browser with JS; SSR builds HTML on the server per request (good SEO); SSG builds HTML once at build time (fastest).

Q13. What is XSS and SQL injection? How to prevent them?

XSS = injected scripts run in other users' browsers → escape output, textContent, CSP. SQL injection = malicious SQL in input → parameterised queries, validation.

Q14. What is Git? Branch vs merge?

Version control system. A branch is a separate line of work; merge joins it back into another branch.

Q15. What is CI/CD?

CI = Continuous Integration (auto build and test on each push). CD = Continuous Delivery / Deployment (auto release to staging / production).

Q16. How do you make a website faster?

Compress / lazy-load images, minify and bundle, caching, CDN, gzip / Brotli, defer scripts, fewer requests, optimise database, SSR / SSG.

Q17. Cookies vs localStorage vs sessionStorage?

Cookies (~4 KB) go to the server with each request; localStorage (~5 MB) is permanent in the browser; sessionStorage lasts until the tab closes.

Q18. Library vs framework? Monolith vs microservices?

You call a library; a framework calls your code. A monolith is one deployable app; microservices are small independent services talking via APIs.

Q19. What is the DOM?

A tree representation of the HTML document that JavaScript can read and change.

Q20. What is a CDN, a PWA, and a WebSocket?

CDN: worldwide file servers. PWA: installable offline-capable web app (manifest + service worker). WebSocket: persistent two-way real-time connection.

Q21. What is middleware?

Code that runs between the request and the response (logging, authentication, parsing JSON, error handling) – used in Express with app.use().

Q22. What is the difference between a web server and an application server?

A web server (Nginx, Apache) serves static files and forwards requests; an application server (Node, Django) runs your business logic and talks to the database.

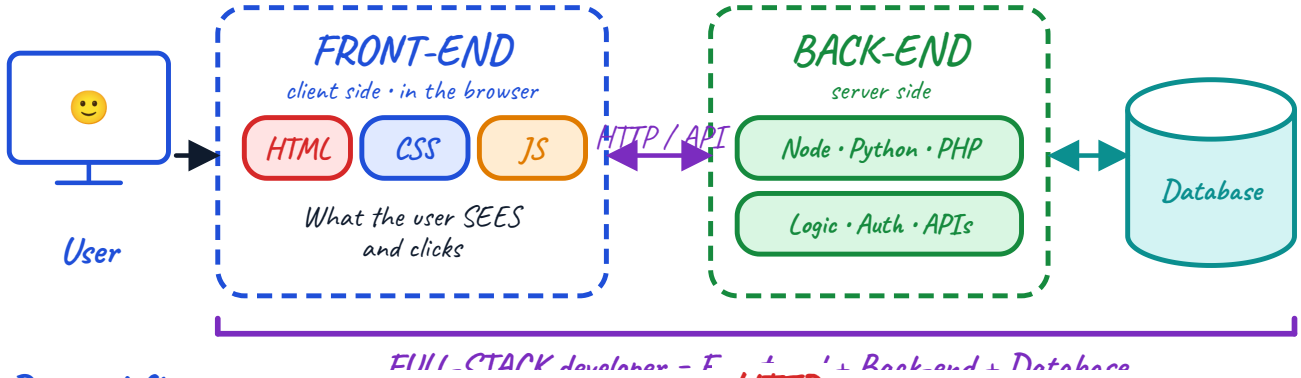
Q23. What is lazy loading and why use it?

Loading images, scripts or pages only when needed (loading="lazy", dynamic import) → faster first load and less data.

Q24. What is Docker?

A tool that packs an app with all its dependencies into a container so it runs the same everywhere (development, testing, production).

24. Quick Cheat Sheet



Request flow

URL → DNS (IP) → TCP + TLS → HTTP request → server + DB → response → browser: DOM + CSSOM → render tree → layout → paint.

HTTP

GET read · **POST** create · **PUT / PATCH** update · **DELETE** remove

2xx ok · 3xx redirect · 4xx client error · 5xx server error

Front-end

HTML (structure) · CSS (Flexbox, Grid, media queries) · JavaScript (DOM, fetch, async) · React / Vue · Tailwind · a11y · responsive · PWA

Back-end

Node + Express / Django / Laravel · routes · middleware · auth (session / JWT / OAuth) · validation · REST / GraphQL · .env

Database

SQL: MySQL, PostgreSQL (tables, JOIN, ACID) · NoSQL: MongoDB, Redis · CRUD · indexes · ORM · backups

Tools

VS Code · DevTools (F12) · terminal · Git & GitHub · npm · Vite · Postman · Figma · Lighthouse · ESLint / Prettier

Security

HTTPS · hash passwords (bcrypt) · escape output (XSS) · parameterised queries (SQLi) · CSRF tokens · CORS · rate limit · npm audit · never commit secrets

Performance & deploy

Compress images · minify · cache · CDN · lazy load · Core Web Vitals (LCP, INP, CLS) · CI/CD · Docker · Netlify / Vercel / VPS / AWS

Final checklist before launching a website ✅

- ➜ Works on mobile, tablet, desktop & many browsers
- ➜ No console errors; all links & forms tested
- ➜ Images optimised; fast (Lighthouse 90+)
- ➜ HTTPS + custom domain connected
- ➜ SEO: title, description, sitemap, alt text
- ➜ Secrets in .env; inputs validated; npm audit clean
- ➜ Favicon, 404 page, analytics, backups
- ➜ Code on GitHub with README

~ Thank you! Keep building ~

by asadokz18 ✿